

THE EFFICIENT KERNEL RUNS SLOW: GPU POWER STATES DECIDE THE LATENCY OF LIGHT VECTOR-SEARCH QUERIES

Akshat Singh Kushwaha

akshatsingh14372@outlook.com

ABSTRACT

Latency comparisons between GPU implementations assume that a kernel’s speed is a property of its code and its hardware. For latency-bound queries it also depends on the power state the driver selects, and the driver selects it from the load the query itself imposes. We study two GPU paths of an exact filtered vector-search engine that return identical top- k results. The *light* path does work proportional to the E eligible rows; the *heavy* path scans all N rows. We ran fresh processes on four GPUs (an RTX 4050 Laptop, RTX 4090s on six cloud hosts, and an H100). The heavy top- k kernel’s device time reproduces within 1% across processes on a machine and within 7% across hosts. The light kernel’s varies up to $4.2\times$ on one machine and $4.6\times$ across hosts with the same GPU model. Continuous telemetry shows why: the driver runs the light path in low performance states (585 MHz SM and 810 MHz memory on the laptop, 645 MHz on the H100) while the heavy path holds boost clocks. In preregistered experiments, an unrelated concurrent GPU load raised the power state and removed the light kernel’s slow tail on every GPU. The share of light runs more than $1.5\times$ slower than their best fell from 52% to 0% on the laptop, 28% to 16% on the 4090s and 11% to 0% on the H100. The same load left the heavy kernel unchanged or slower. Contention alone cannot produce that pattern; a power-state account predicts it. End-to-end effects were small or negative, because the extra load also competes for the GPU and the host. Unlocked-clock latency benchmarks of light GPU work therefore partly measure the governor. They understate the more efficient implementation by a host-dependent factor, and their conclusions transfer poorly across machines. All protocols, raw per-call data and telemetry are released.

1 INTRODUCTION

A retrieval call in an agent or RAG pipeline is often small: one query, a metadata filter that leaves a few thousand candidates, $k = 10$. Whether such a call should run on a GPU, and in which form, is usually decided by measuring it. This paper shows that for such light calls the measurement is not only a property of the implementation and the hardware. It also depends on a hidden variable, the GPU’s performance state, which the driver chooses from the load the call itself imposes.

The observation that started this study was a reproducibility asymmetry. Our system has two GPU execution paths for exact filtered top- k search that return identical results. A *light* path uploads the list of E eligible rows and does device work proportional to E . A *heavy* path evaluates the filter inside the kernel and does device work proportional to all N rows, up to $1,000\times$ more at our most selective filter. We re-ran the same query cell in fresh processes. The heavy path’s top- k kernel time reproduced to three decimal places, while the light path’s jumped between modes $2.4\times$ apart (Section 4), and the fast mode coincided with higher GPU clocks. On other hosts with the same GPU model, the light kernel was up to $3.8\times$ faster in median, while the heavy kernel differed by at most 4%.

We test the explanation directly. Clock locking needs administrator rights on consumer machines and was refused inside cloud containers. So instead of locking the clock, we raise it: a separate process, the *heater*, keeps the GPU lightly busy. The heater can only compete with the measured

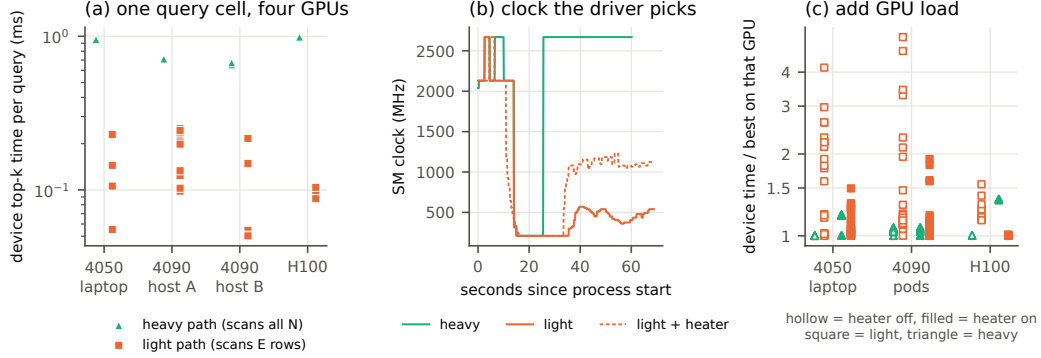


Figure 1: **Work-efficient GPU queries run at whatever power state the driver gives them.** (a) Device time of the top- k kernel for one query cell ($N = 100K$, $E = 3,000$, $D = 384$, $k = 10$, batch 1), one point per fresh process. The heavy path does $33\times$ more work and reproduces within 1% on a machine and 7% across hosts; the light path varies up to $4.2\times$ on a machine and $4.3\times$ across RTX 4090 hosts. (b) SM clock sampled every 50 ms during one laptop process per condition ($E = 10,000$), including data loading. The driver holds the heavy path at boost and drops the light path to ≈ 500 MHz; an unrelated concurrent GPU load (the *heater*) raises it. (c) Light (squares) and heavy (triangles) device time relative to the best value observed for that GPU and E , heater off (hollow) versus on (filled), one point per block and E . The heater removes the light kernel’s slow tail on every GPU and only slows the heavy kernel.

query, so under a pure contention account it slows every path. Under a power-state account it speeds up whatever was running below its attainable clock and slows only what was already at boost. Across preregistered experiments on three GPU types (Figure 1c, Section 6), the second pattern appears on every GPU. With the heater, the share of light-kernel runs more than $1.5\times$ slower than their best falls from 52% to 0% on the laptop, from 28% to 16% on the RTX 4090s and from 11% to 0% on the H100. The heavy kernel never speeds up. End-to-end effects are smaller and mixed, because the heater also competes for the GPU and, on some hosts, for the CPU. We report both.

Contributions.

- **A reproducibility asymmetry.** For identical results, the more work-efficient GPU kernel is the less reproducible one. It varies up to $4.2\times$ on a machine and $4.6\times$ across same-model hosts, while a kernel doing up to $1,000\times$ more work reproduces within 1% on a machine and 7% across hosts (Section 4).
- **Mechanism and intervention.** Continuous telemetry ties the variation to driver-selected performance states. A preregistered, contention-conservative intervention with a built-in negative control shows that raising the power state removes the light kernel’s slow tail on three GPU types while never speeding up the heavy kernel (Sections 5–6).
- **Consequences for evaluation.** Unlocked-clock comparisons between implementations of different intensity understate the efficient one by a host-dependent factor ($3.4\times$ to $12.6\times$ light-over-heavy advantage for the same cell across our hosts). Kernel-level conclusions do not transfer between machines with the same GPU. We give reporting rules (Section 7).

2 THE QUERY PATHS AND WHY THEIR LOAD DIFFERS

We study Qenlo, an embedded Rust vector store that answers exact (exhaustive) filtered cosine top- k queries over a canonical collection of N normalized FP32 vectors of dimension D with per-row metadata. A predicate selects E eligible rows; the query returns the k eligible rows nearest to the query. Every execution path below returns the same result set (all runs pass an FP64-oracle

recall@10 gate of 0.99; observed recall is 1.0 except where noted), so choosing a path is a pure latency decision.

CPU scan. A single-threaded AVX2 scan over the eligible rows. For $E \leq 4,096$ it scores in FP32 and rescores boundary candidates in FP64; above that it accumulates in FP64 throughout. The kernel switch is part of the implementation and applies equally to every condition we compare.

GPU paths (WGPU/WGSL). Vectors, identifiers and metadata are resident on the device. Each query uploads the query vector and an eligibility description, runs a scoring kernel and a top- k selection kernel, and blocks on a readback of k candidates, which the host merges. The paths differ only in how eligibility reaches the device:

- *Compact rows (the light path).* The host materializes the sorted list of E eligible row ids and uploads $4E$ bytes. Scoring and selection touch only those E rows, so device work is proportional to E .
- *Shader predicate (the heavy path).* The host uploads per-row liveness ($4N$ bytes) and the predicate bounds; the kernels run over all N rows and evaluate the predicate in the shader. Selection scans all N scores k times in one workgroup, so device work is proportional to N regardless of E .

The heavy path therefore does up to N/E times more device work for the same answer ($1,000\times$ at $E = 100$, $N = 10^5$). This asymmetry is the lever of the paper: the two paths put very different loads on the GPU while computing identical results.

Timing boundary. Latency is measured from the host search call to host-visible ordered results, including eligibility preparation, upload, dispatch, synchronization, readback and merge. Device-side scoring and selection times come from GPU timestamp queries in the same calls; they overlap host phases and are not additive. Host row materialization is timed separately on the CPU.

3 EXPERIMENTAL DESIGN

Workload. 100,000 AG News sentence embeddings (all-MiniLM-L12-v2, $D = 384$; Nielsen, 2023) form the corpus; 5,000 disjoint held-out embeddings are the queries. Synthetic unique timestamps are attached to rows and a timestamp range selects exactly $E \in \{100, 1K, 3K, 10K, 30K\}$ eligible rows (0.1%–30%). Batch size is 1 and $k = 10$, the regime of an interactive retrieval or agent-memory lookup.

Unit of replication. Every condition runs in a fresh process: dataset load, 200 warm-up queries, then three passes over the 5,000 queries (15,000 timed calls). The per-process statistic is the lower median of the three per-pass percentiles. A *block* runs every condition once in a shuffled order; blocks are the independent replicates. Paired effects are computed within a block and summarized by the median over blocks of per-block log-ratios, with 95% percentile-bootstrap intervals over blocks (20,000 draws). The 15,000 calls within a process are repeated measurements and are never treated as independent.

Telemetry. `nvidia-smi` samples SM clock, memory clock, performance state (P-state), utilization and power every 50 ms for the whole life of each process.

Intervention: a GPU heater. To move the GPU’s power state without touching the measured process, a separate process (the *heater*) repeatedly dispatches a small compute kernel (64 workgroups $\times 256$ threads, 500 dependent FMAs each), waits for it, and sleeps 1 ms. Its own reported utilization is a few percent. The heater can only *compete* with the measured process for the GPU, so any speed-up it produces is conservative evidence that the measured process was limited by power state rather than by capacity. The heavy path, which already holds a high P-state, is the built-in negative control: under a pure contention account the heater slows both paths, while under a power-state account it speeds up the light path and slows only the heavy one.

Table 1: Top- k kernel device time per query (ms): median over fresh heater-off processes (min-max), and max/min across processes. Heavy = shader predicate over all $N = 100K$ rows; light = compact eligible rows. RTX 4090 host A is the retained September 24 archive (one host, 5 blocks); pods B are five September 25 pods on five different hosts, one block each. Identical binary source, dataset bytes and queries.

E	GPU	n	Heavy (ms)	max/min	Light (ms)	max/min
1,000	RTX 4050 Laptop	5	0.950 (0.949–0.954)	1.01	0.030 (0.030–0.030)	1.00
1,000	RTX 4090 host A	5	0.711 (0.711–0.711)	1.00	0.053 (0.051–0.065)	1.27
1,000	RTX 4090 pods B	5	0.681 (0.650–0.695)	1.07	0.028 (0.027–0.056)	2.08
1,000	H100 SXM	3	0.987 (0.982–0.987)	1.00	0.039 (0.039–0.039)	1.02
3,000	RTX 4050 Laptop	5	0.952 (0.951–0.956)	1.01	0.106 (0.055–0.230)	4.16
3,000	RTX 4090 host A	5	0.711 (0.711–0.711)	1.00	0.210 (0.098–0.251)	2.56
3,000	RTX 4090 pods B	5	0.685 (0.652–0.700)	1.07	0.055 (0.050–0.216)	4.30
3,000	H100 SXM	3	0.988 (0.988–0.988)	1.00	0.094 (0.088–0.104)	1.18
10,000	RTX 4050 Laptop	5	0.941 (0.940–0.941)	1.00	0.461 (0.425–0.595)	1.40
10,000	RTX 4090 host A	5	0.703 (0.703–0.703)	1.00	0.440 (0.337–0.512)	1.52
10,000	RTX 4090 pods B	5	0.673 (0.643–0.687)	1.07	0.142 (0.128–0.589)	4.61
10,000	H100 SXM	3	0.977 (0.972–0.977)	1.01	0.205 (0.188–0.206)	1.10

Hosts and preregistration. (i) A Windows 11 laptop (Intel i5-13420H, RTX 4050 Laptop GPU, driver 616.56, Vulkan, AC power). (ii) Five cloud pods with an RTX 4090 each, on five different hosts (Linux, Vulkan, drivers 570.195 and 580.126–580.173), one block per pod, so host-to-host differences enter only between-block variance. (iii) One H100 SXM pod (Linux, Vulkan) running a reduced design. Each protocol, with its predictions and refutation criteria, was written and hashed before its data were collected (Appendix A). Clock locking (`nvidia-smi -lgc`) was not available: it requires administrator rights on the laptop and was refused inside the cloud containers.

4 HEAVY KERNELS REPRODUCE; LIGHT KERNELS DO NOT

Table 1 reports the median device time of the top- k selection kernel per fresh process, for both paths at three selectivities on four GPUs. We use device timestamps rather than end-to-end latency because they isolate the GPU from host-side effects; Section 6 returns to end-to-end latency.

Three facts stand out. First, the heavy kernel reproduces within 1% between processes on any single machine (the laptop, 4090 host A and the H100). Across the five pod hosts, which span two driver branches (570 and 580), it stays within 7%. Second, the light kernel does not. Its spread between processes reaches $2.6\times$ on 4090 host A and $4.2\times$ on the laptop, and $4.3\text{--}4.6\times$ across the five 4090 pod hosts. At $E = 3,000$ its median differs $3.8\times$ between host A and the pods (0.210 versus 0.055 ms). Third, the light path is still faster than the heavy path in every cell: the asymmetry lies in its reproducibility, not its direction.

The retained archive shows the same asymmetry within one host over time. Across ten identical blocks, the light path’s selection time spans $2.44\times$ and correlates with the GPU clock recorded at process end (Spearman $\rho = -0.83$, $p = 0.003$). The heavy kernels in the same archive vary by at most 0.1% wherever their clock was at boost.

5 WHICH POWER STATE EACH PATH GETS

Table 2 reports the median SM and memory clocks while the GPU was active (samples with non-zero utilization) for each path. On every GPU the heavy path holds boost SM clocks: 2,670–2,820 MHz on the consumer GPUs and 1,980 MHz on the H100. The light path runs lower. On the laptop at $E = 10,000$ the driver also cuts the memory clock from 8,201 to 810 MHz (performance state P5), a $10\times$ cut in bandwidth for a memory-bound scoring kernel. On the H100 the light path’s SM clock falls from 1,605 to 645 MHz between $E = 1,000$ and 10,000.

Table 2: Median SM / memory clock (MHz) while the GPU is active, pooled over blocks. Clocks are sampled by `nvidia-smi` every 50 ms, which is itself a partial sampler (Yang et al., 2023).

GPU	E	Heavy path	Light path	Light path + heater
RTX 4050 Laptop	1,000	2670 / 8201	2130 / 8201	2130 / 8201
RTX 4050 Laptop	3,000	2670 / 7001	1080 / 7001	1912 / 8201
RTX 4050 Laptop	10,000	2670 / 8201	585 / 810	1215 / 8201
RTX 4090 pods B	1,000	2820 / 10501	2520 / 10501	2550 / 10501
RTX 4090 pods B	3,000	2820 / 10501	2505 / 10501	2550 / 10501
RTX 4090 pods B	10,000	2820 / 10501	2235 / 810	2520 / 10501
H100 SXM	1,000	1980 / 2619	1605 / 2619	1980 / 2619
H100 SXM	3,000	1980 / 2619	1275 / 2619	1980 / 2619
H100 SXM	10,000	1980 / 2619	645 / 2619	1980 / 2619

Table 3: Device time relative to the best observed for that GPU and E : median, 90th percentile, and share of runs more than $1.5\times$ slower than best, heater off versus on. One observation per block and E .

Path	GPU	n	Heater off			Heater on		
			median	p90	$> 1.5\times$	median	p90	$> 1.5\times$
Light	RTX 4050 Laptop	25	1.59	2.61	52%	1.07	1.27	0%
Light	RTX 4090 pods	25	1.15	3.29	28%	1.06	1.59	16%
Light	H100 SXM	9	1.24	1.41	11%	1.00	1.01	0%
Heavy	RTX 4050 Laptop	25	1.00	1.00	0%	1.19	1.20	0%
Heavy	RTX 4090 pods	25	1.05	1.07	0%	1.05	1.07	0%
Heavy	H100 SXM	9	1.00	1.01	0%	1.36	1.37	0%

We do not know what the governor responds to, so we state only what we observe. The heavy path spends 52–84% of each call inside its two kernels and always holds boost. The light path spends 13–43% and does not. Within the light path, however, the clock *falls* as E grows on the laptop and the H100, even though per-call device work grows. A simple busy-fraction account does not explain this. The busy fraction is also endogenous, because a lower clock lengthens the kernels that define it. The NVIDIA policy is undocumented (Spaan et al., 2026), and our claims concern its observable consequences.

6 INTERVENTION: MAKING THE GPU BUSIER

Each experiment ran every condition (engine $\times E \times$ heater on/off) once per block, in shuffled order and fresh processes (Section 3). Table 3 gives the most direct test. For every block and E , it divides each path’s device time by the best value observed for that GPU and E in either arm. Both arms share the same reference, so a heater-off run that happened to be slow cannot flatter the heater through regression to the mean.

Device time: the predicted interaction appears on every GPU. Without the heater, the light kernel runs a median 1.15–1.59 $\times$ slower than its best, with a tail reaching 3.3 $\times$. With the heater, its median is 1.00–1.07 and slow runs nearly disappear. The heavy kernel has no tail to remove. With the heater it becomes uniformly slower on the laptop (1.19 $\times$) and the H100 (1.36 $\times$), the cost of sharing the GPU, and stays unchanged on the 4090 pods. The heater’s effect on the light kernel also grows with how slow that kernel was: across 59 block-level pairs, the heater-on/off ratio correlates with the heater-off slowdown (Spearman $\rho = -0.66$), and pairs that were at least 1.5 $\times$ slow were cut to a median 0.47. Unlike Table 3, this correlation is exposed to regression to the mean, so we report it only as consistent with the account. Paired ratios with block-bootstrap intervals are in Appendix Table 5.

End to end: small and host-dependent. The heater also competes for GPU submission and read-back, and on some hosts for the CPU: on the H100 pod it slowed host row materialization by 40–48%. End to end, the light path’s median call latency with the heater was 0.94–0.96 of its heater-off value on the 4090 pods at $E = 3,000$ –10,000 (1.07 at 30,000) and 0.95–0.97 on the laptop at $E \geq 3,000$, with intervals that mostly include 1. On the H100 it was 1.10–1.24. Heavy calls were 6–9% slower on the consumer GPUs and 25–27% slower on the H100. Under the same added load the light path fared better than the heavy path in 10 of 13 cells, but whether it gained or lost overall depends on how much contention the host adds. The $1.9\times$ end-to-end speed-up in our laptop pilot did not replicate reliably across blocks.

Scorecard against the preregistered predictions (Appendix Table 4).

- P1 predicted a light-path end-to-end speed-up of at least 10% at $E \leq 3,000$. It *failed* on every GPU.
- P2 predicted no heavy-path speed-up. It held in every cell.
- P3 predicted that the light path’s ratio would fall below the heavy path’s. It held in every cell for device time and in 10 of 13 cells end to end.
- P4 predicted that light device time falls while host materialization stays within 10%. It held in 6 of 13 cells and failed where the heater also slowed the host.

The device-level predictions, which isolate the GPU, held. The end-to-end prediction did not.

7 CONSEQUENCES

Benchmarks understate the efficient implementation. A benchmark that compares a work-efficient GPU kernel with a less efficient one at low load runs the efficient one at a lower clock. Here the light path won every cell regardless. But its measured device-time advantage over the heavy path at $E = 3,000$ is $3.4\times$ on 4090 host A, $9.0\times$ on the laptop, $10.5\times$ on the H100 and $12.6\times$ on the 4090 pods, for identical code. Optimizations that remove GPU work (tighter filtering, smaller kernels, early exit) are measured at a discount the benchmark cannot see, and each machine applies a different discount.

Kernel-level conclusions do not transfer between machines with the same GPU. Six RTX 4090 hosts gave light-kernel medians spanning $3.8\times$ for the same cell. Any threshold or cost model calibrated on one of them, for example a CPU/GPU router, encodes that host’s power behavior. The study that preceded this one fitted a scalar CPU/GPU routing threshold with zero errors on 31 development pairs. On a preregistered held-out suite it chose the slower route in 8 of 16 workloads, several of them within the process-to-process variation measured here (Appendix C). In our data the heater never changed a CPU/GPU winner (Appendix Table 6). We therefore do not claim that power state moves this system’s crossover, only that it moves the margins a router is calibrated on.

Isolated benchmarks misstate co-located latency. In deployment a retrieval kernel often shares the GPU with a model; our heater is a crude stand-in for that load. On an idle GPU the light path is exposed to deep power states; under load, to contention. The heavy path is exposed only to contention. Latency measured on an idle GPU is therefore not latency under co-location, and the error differs between implementations.

What to report. For latency-bound GPU work:

- Replicate across fresh processes and report the spread between them; repetitions inside one process share its power state.
- Log clocks and performance state continuously, not just before and after a run.
- Report device-timestamp kernel time alongside end-to-end latency, to separate power-state effects from host effects.
- Where clocks cannot be locked, add a load-raising control such as the heater, together with a heavy negative control.
- When comparing implementations, state whether they ran in the same power state.

8 RELATED WORK

Power management versus latency. On servers, low-power states and slow frequency governors are known to inflate the latency of latency-critical services at low load (Meisner et al., 2011; Kanev et al., 2014), motivating latency-aware controllers (Lo et al., 2014; Kasture et al., 2015; Hsu et al., 2015; Chou et al., 2019). Mobile GPU governors mispredict bursty and concurrent work (Pathania et al., 2014; Lin et al., 2023; Zhang et al., 2025), and at low locked clocks desktop GPUs can drop into idle-like states (Adamek et al., 2021). GPU DVFS work for machine learning treats frequency as a knob to *lower* for energy (Tang et al., 2019; You et al., 2023; Chung et al., 2024; Qiu et al., 2024; Stojkovic et al., 2025; Kakolyris et al., 2025; Maliakel et al., 2025; Spaan et al., 2026). We study the opposite situation: the default policy under-clocks latency-bound work that the application would gladly run fast. We are not aware of a report that this reverses the ranking of two implementations of the same GPU operation, or that an unrelated co-running GPU load speeds a latency-bound GPU workload up.

Benchmark validity. Innocuous setup details bias measured speed-ups (Mytkowicz et al., 2009); variation across executions must be sampled and reported (Kalibera & Jones, 2013; Maricq et al., 2018). For GPUs, power management causes 8–22% variation across same-model devices under load (Sinha et al., 2022) and 1–14% in GEMM timings (Golden et al., 2025); kernel benchmarking practice therefore often locks clocks (Oliaro et al., 2026; Chu & Li, 2026), and `nvidia-smi` telemetry itself is sampled only part-time (Yang et al., 2023). Our effect is an order of magnitude larger than these variations and has a sign that favors the less efficient implementation.

Filtered vector search and CPU/GPU placement. Filtered nearest-neighbor search is an active area (Gollapudi et al., 2023; Patel et al., 2024; Iff et al., 2026; Shi et al., 2025; Li et al., 2025; Amanbayev et al., 2026), including GPU systems (Johnson et al., 2021; Xi et al., 2025). Whether to run vector search on a GPU at all depends on data movement and batch size (Gregg & Hazelwood, 2011; Mageirakos et al., 2026), and cost-model-based CPU/GPU operator placement is known to be fragile (Breß et al., 2016). The choice between an eligible-id list and a full-width filter mirrors selection vectors versus bitmaps in vectorized databases (Ngom et al., 2021). Our contribution to this line is that the measured margins between such alternatives depend on the GPU’s power state, which the alternatives themselves influence.

9 LIMITATIONS

One system, one workload family. Both paths come from one engine with a simple selection kernel; heavier or better-tuned kernels (e.g. FAISS or cuVS selection) may sit in different states. We claim the mechanism, not these magnitudes. *No clock locking.* The decisive direct test, re-running both paths at identical locked clocks, was unavailable: it needs administrator rights on the laptop and was refused in the cloud containers. The heater is an indirect, contention-conservative substitute. *Telemetry.* `nvidia-smi` samples every 50 ms and only part-time (Yang et al., 2023); clocks are reported for active samples of a whole process, including warm-up. *Small n.* Five blocks on the laptop, five RTX 4090 pods (one block each, on five different hosts), three H100 blocks; intervals are coarse, and pod-to-pod host differences widen them. One pod was replaced before analysis because its host CPU was about 14× slower single-threaded; its partial block is retained. *Host coupling.* On the laptop the heater also changed host-side timings, so end-to-end effects there are not attributable to the GPU clock alone. *API.* All runs use Vulkan through WGPU; CUDA runtimes may be managed differently. *Driver versions.* 616.56 (Windows) and 570.195–580.173 (Linux).

10 CONCLUSION

The fastest implementation of a light GPU query is also the one whose speed depends most on the machine it happens to run on. Its latency is partly set by a governor reacting to its own light load. The heavy alternative gets a fast clock by wasting work, and so looks both more stable and relatively better than it is. Benchmarks of retrieval kernels, and the routers and design decisions built on them, should treat the GPU’s power state as an experimental variable: record it, replicate across it, and, when it cannot be controlled, perturb it.

REPRODUCIBILITY

Protocols with predictions were written and hashed before data collection (Appendix A). Raw per-call samples, 50ms telemetry, analysis scripts and the heater are in the repository: `research/data/raw/2026-09-25-dvfs-*`, `research/scripts/analyze_dvfs_dl.py`, `cross_host.py`, `pool_hosts.py`, `crates/qenlo/examples/gpu_heater.rs`. Tables and figures are regenerated by `paper/v2/scripts/`. Cloud compute for all new runs cost about US\$6.

REFERENCES

- Karel Adamek, Jan Novotny, Jeyarajan Thiagalingam, and Wesley Armour. Efficiency near the edge: Increasing the energy efficiency of ffts on gpus for real-time edge computing. *IEEE Access*, 9:18167–18182, 2021. ISSN 2169-3536. doi: 10.1109/access.2021.3053409. URL <http://dx.doi.org/10.1109/ACCESS.2021.3053409>.
- Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. Filtered approximate nearest neighbor search in vector databases: System design and performance analysis. *arXiv preprint arXiv:2602.11443*, 2026. URL <https://arxiv.org/abs/2602.11443>.
- Sebastian Breß, Henning Funke, and Jens Teubner. Robust query processing in co-processor-accelerated databases. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD/PODS’16*, pp. 1891–1906. ACM, June 2016. doi: 10.1145/2882903.2882936. URL <http://dx.doi.org/10.1145/2882903.2882936>.
- Chih-Hsun Chou, Laxmi N. Bhuyan, and Daniel Wong. μ DPM: Dynamic power management for the microsecond era. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 120–132. IEEE, February 2019. doi: 10.1109/hpca.2019.00032. URL <http://dx.doi.org/10.1109/HPCA.2019.00032>.
- Logan Chu and Dong Li. Hardware-attributed operator profiling for pytorch, 2026. URL <https://arxiv.org/abs/2609.11938>.
- Jae-Won Chung, Yile Gu, Insu Jang, Luoxi Meng, Nikhil Bansal, and Mosharaf Chowdhury. Reducing energy bloat in large model training. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP ’24*, pp. 144–159. ACM, November 2024. doi: 10.1145/3694715.3695970. URL <http://dx.doi.org/10.1145/3694715.3695970>.
- Alicia Golden, Michael Kuchnik, Samuel Hsia, Zachary DeVito, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. Prism: Probabilistic runtime insights and scalable performance modeling for large-scale distributed training, 2025. URL <https://arxiv.org/abs/2510.15596>.
- Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pp. 3406–3416. Association for Computing Machinery, 2023. doi: 10.1145/3543507.3583552. URL <https://doi.org/10.1145/3543507.3583552>.
- Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate cpu vs. gpu performance without the answer. In *(IEEE ISPASS) IEEE INTERNATIONAL SYMPOSIUM ON PERFORMANCE ANALYSIS OF SYSTEMS AND SOFTWARE*, pp. 134–144. IEEE, April 2011. doi: 10.1109/ispass.2011.5762730. URL <http://dx.doi.org/10.1109/ISPASS.2011.5762730>.
- Chang-Hong Hsu, Yunqi Zhang, Michael A. Laurenzano, David Meisner, Thomas Wenisch, Jason Mars, Lingjia Tang, and Ronald G. Dreslinski. Adrenaline: Pinpointing and reining in tail queries with quick voltage boosting. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, pp. 271–282. IEEE, February 2015. doi: 10.1109/hpca.2015.7056039. URL <http://dx.doi.org/10.1109/HPCA.2015.7056039>.

- Patrick Iff, Paul Brügger, Marcin Chrapek, David Kochergin, Maciej Besta, and Torsten Hoefer. Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 610–622, 2026. doi: 10.1145/3805712.3809731. URL <https://doi.org/10.1145/3805712.3809731>.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572. URL <https://doi.org/10.1109/TBDATA.2019.2921572>.
- Andreas Kosmas Kakolyris, Dimosthenis Masouros, Petros Vavaroutsos, Sotirios Xydis, and Dimitrios Soudris. throttlem: Predictive gpu throttling for energy efficient llm inference serving. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 1363–1378. IEEE, March 2025. doi: 10.1109/hpca61900.2025.00103. URL <http://dx.doi.org/10.1109/HPCA61900.2025.00103>.
- Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the 2013 international symposium on memory management, ISMM '13*, pp. 63–74. ACM, June 2013. doi: 10.1145/2464157.2464160. URL <http://dx.doi.org/10.1145/2464157.2464160>.
- Svilen Kanev, Kim Hazelwood, Gu-Yeon Wei, and David Brooks. Tradeoffs between power management and tail latency in warehouse-scale applications. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*, pp. 31–40. IEEE, October 2014. doi: 10.1109/iiswc.2014.6983037. URL <http://dx.doi.org/10.1109/IISWC.2014.6983037>.
- Harshad Kasture, Davide B. Bartolini, Nathan Beckmann, and Daniel Sanchez. Rubik: fast analytical power management for latency-critical systems. In *Proceedings of the 48th International Symposium on Microarchitecture, MICRO-48*, pp. 598–610. ACM, December 2015. doi: 10.1145/2830772.2830797. URL <http://dx.doi.org/10.1145/2830772.2830797>.
- Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proceedings of the ACM on Management of Data*, 3(6):298:1–298:26, 2025. doi: 10.1145/3769763. URL <https://doi.org/10.1145/3769763>.
- Chengdong Lin, Kun Wang, Zhenjiang Li, and Yu Pu. A workload-aware dvfs robust to concurrent tasks for mobile devices. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking, MobiCom '23*, pp. 1–16. ACM, July 2023. doi: 10.1145/3570361.3592524. URL <http://dx.doi.org/10.1145/3570361.3592524>.
- David Lo, Liqun Cheng, Rama Govindaraju, Luiz Andre Barroso, and Christos Kozyrakis. Towards energy proportionality for large-scale latency-critical workloads. In *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, pp. 301–312. IEEE, June 2014. doi: 10.1109/isca.2014.6853237. URL <http://dx.doi.org/10.1109/ISCA.2014.6853237>.
- Vasilis Mageirakos, Joel André, Marko Kabić, Bowen Wu, Yannis Chronis, and Gustavo Alonso. To gpu or not to gpu: Vector search in relational engines, 2026. URL <https://arxiv.org/abs/2605.15957>.
- Paul Joe Maliakel, Shashikant Ilager, and Ivona Brandic. Characterizing llm inference energy-performance tradeoffs across workloads and gpu scaling, 2025. URL <https://arxiv.org/abs/2501.08219>.
- Aleksander Maricq, Dmitry Duplyakin, Ivo Jimenez, Carlos Maltzahn, Ryan Stutsman, and Robert Ricci. Taming performance variability. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pp. 409–425, Carlsbad, CA, October 2018. USENIX Association. ISBN 978-1-939133-08-3. URL <https://www.usenix.org/conference/osdi18/presentation/maricq>.
- David Meisner, Christopher M. Sadler, Luiz André Barroso, Wolf-Dietrich Weber, and Thomas F. Wenisch. Power management of online data-intensive services. In *Proceedings of the 38th annual international symposium on Computer architecture, ISCA '11*, pp. 319–330. ACM, June

2011. doi: 10.1145/2000064.2000103. URL <http://dx.doi.org/10.1145/2000064.2000103>.
- Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th international conference on Architectural support for programming languages and operating systems*, ASPLOS09, pp. 265–276. ACM, March 2009. doi: 10.1145/1508244.1508275. URL <http://dx.doi.org/10.1145/1508244.1508275>.
- Amadou Ngom, Prashanth Menon, Matthew Butrovich, Lin Ma, Wan Shen Lim, Todd C. Mowry, and Andrew Pavlo. Filter representation in vectorized query execution. In *Proceedings of the 17th International Workshop on Data Management on New Hardware (DaMoN 2021)*, SIGMOD/PODS '21, pp. 1–7. ACM, June 2021. doi: 10.1145/3465998.3466009. URL <http://dx.doi.org/10.1145/3465998.3466009>.
- Beatrix Miranda Ginn Nielsen. DTU pretrained sentence BERT AG News embeddings, 2023. URL <https://doi.org/10.11583/DTU.21286923.v1>.
- Gabriele Oliaro, Yichao Fu, May Jiang, Owen Lu, Junli Wang, Zhihao Jia, Hao Zhang, and Samyam Rajbhandari. Fastkernels: Benchmarking gpu kernel generation in production, 2026. URL <https://arxiv.org/abs/2605.23215>.
- Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3):120:1–120:27, 2024. doi: 10.1145/3654923. URL <https://doi.org/10.1145/3654923>.
- Anuj Pathania, Qing Jiao, Alok Prakash, and Tulika Mitra. Integrated cpu-gpu power management for 3d mobile games. In *Proceedings of the 51st Annual Design Automation Conference, DAC '14*, pp. 1–6. ACM, June 2014. doi: 10.1145/2593069.2593151. URL <http://dx.doi.org/10.1145/2593069.2593151>.
- Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K. Iyer. Power-aware deep learning model serving with μ -Serve. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 75–93, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-41-0. URL <https://www.usenix.org/conference/atc24/presentation/qiu>.
- Jiayang Shi, Yuzheng Cai, and Weiguo Zheng. Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study. *arXiv preprint arXiv:2509.07789*, 2025. URL <https://arxiv.org/abs/2509.07789>.
- Prasoon Sinha, Akhil Guliani, Rutwik Jain, Brandon Tran, Matthew D. Sinclair, and Shivararam Venkataraman. Not all gpus are created equal: Characterizing variability in large-scale, accelerator-rich systems. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 01–15. IEEE, November 2022. doi: 10.1109/sc41404.2022.00070. URL <http://dx.doi.org/10.1109/SC41404.2022.00070>.
- Jeffrey Spaan, Kuan-Hsun Chen, and Ana-Lucia Varbanescu. Reducing compute waste in llms through kernel-level dvfs, 2026. URL <https://arxiv.org/abs/2601.08539>.
- Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 1348–1362. IEEE, March 2025. doi: 10.1109/hpca61900.2025.00102. URL <http://dx.doi.org/10.1109/HPCA61900.2025.00102>.
- Zhenheng Tang, Yuxin Wang, Qiang Wang, and Xiaowen Chu. The impact of gpu dvfs on the energy and performance of deep learning: an empirical study. In *Proceedings of the Tenth ACM International Conference on Future Energy Systems, e-Energy '19*, pp. 315–325. ACM, June 2019. doi: 10.1145/3307772.3328315. URL <http://dx.doi.org/10.1145/3307772.3328315>.

Jingyi Xi, Chenghao Mo, Benjamin Karsin, Artem Chirkin, Mingqin Li, and Minjia Zhang. Vecflow: A high-performance vector data management system for filtered-search on gpus, 2025. URL <https://arxiv.org/abs/2506.00812>.

Zeyu Yang, Karel Adamek, and Wesley Armour. Part-time power measurements: nvidia-smi’s lack of attention, 2023. URL <https://arxiv.org/abs/2312.02741>.

Jie You, Jae-Won Chung, and Mosharaf Chowdhury. Zeus: Understanding and optimizing GPU energy consumption of DNN training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pp. 119–139, Boston, MA, April 2023. USENIX Association. ISBN 978-1-939133-33-5. URL <https://www.usenix.org/conference/nsdi23/presentation/you>.

Zongpu Zhang, Pranab Dash, Y. Charlie Hu, Qiang Xu, Jian Li, and Haibing Guan. Dissecting the impact of mobile dvfs governors on llm inference performance and energy efficiency, 2025. URL <https://arxiv.org/abs/2507.02135>.

A PROTOCOLS, PREDICTIONS AND DEVIATIONS

Four protocols were written and SHA-256 hashed before their data existed: the laptop experiment D1 (`research/experiments/dvfs-d1/protocol.md`), the RTX 4090 pod replication D1R (`dvfs-d1r/`), the H100 arm (`dvfs-h100/`) and an API/batch follow-up D2 (`dvfs-d2/`, not yet run). One exploratory pilot preceded them (`research/data/raw/2026-09-25-dvfs-pilot/`) and is used only to motivate the design. D1 predicted, for batch-1 queries: P1, the heater lowers light-path end-to-end P50 by at least 10% (ratio < 0.9 , interval excluding 1) at $E \leq 3,000$; P2, the heater does not speed up the heavy path (ratio ≥ 0.95); P3, the light path’s ratio is below the heavy path’s; P4, the heater lowers light-path device time while host row materialization stays within $\pm 10\%$. The same rules were applied unchanged to every host.

Table 4: Preregistered predictions by GPU and E . P3 is shown for end-to-end P50 / device time. “n/a”: P1 was registered only for $E \leq 3,000$.

GPU	E	P1 call < 0.9	P2 heavy ≥ 0.95	P3 light $<$ heavy	P4 device $\downarrow$, host flat
RTX 4050 Laptop	100	no	yes	no / yes	no
RTX 4050 Laptop	1,000	no	yes	no / yes	no
RTX 4050 Laptop	3,000	no	yes	yes / yes	no
RTX 4050 Laptop	10,000	n/a	yes	yes / yes	no
RTX 4050 Laptop	30,000	n/a	yes	yes / yes	yes
RTX 4090 pods B	100	no	yes	yes / yes	no
RTX 4090 pods B	1,000	no	yes	yes / yes	yes
RTX 4090 pods B	3,000	no	yes	yes / yes	yes
RTX 4090 pods B	10,000	n/a	yes	yes / yes	yes
RTX 4090 pods B	30,000	n/a	yes	no / yes	yes
H100 SXM	1,000	no	yes	yes / yes	no
H100 SXM	3,000	no	yes	yes / yes	no
H100 SXM	10,000	n/a	yes	yes / yes	yes

P1 fails everywhere at the block level; the pilot’s $1.9\times$ end-to-end speed-up on the laptop did not replicate reliably across blocks. P2 and the device-time half of P3 hold in every cell. P4 fails where the heater also slowed host-side work. On the H100 pod, host row materialization became 40–48% slower with the heater running, which shows that the heater process took host CPU time as well as GPU time. The end-to-end slowdowns on that host are therefore not evidence against the power-state account. They show that our heater is not a clean GPU-only perturbation.

Deviations. (i) The clock summary in the preregistered analysis script took the median over the whole process, including data loading; we added a GPU-active median (utilization > 0) before inspecting any cross-host comparison and report both in the released CSVs. (ii) Two pod setup attempts failed because of script bugs (a missing dataset copy, a mis-specified pod) and were re-run

Table 5: Paired heater-on / heater-off ratios (median over blocks of per-block ratios, 95% block-bootstrap interval). Below 1 = faster with the heater. Device = top- k kernel time; call = end-to-end P50. n = blocks.

GPU	E	n	Light device	Heavy device	Light call	Heavy call
RTX 4050 Laptop	100	5	0.91 [0.90, 1.12]	1.19 [1.00, 1.20]	1.11 [0.78, 1.80]	1.09 [1.02, 1.12]
RTX 4050 Laptop	1,000	5	1.00 [0.98, 1.01]	1.18 [1.18, 1.19]	1.13 [1.05, 1.64]	1.06 [1.04, 1.09]
RTX 4050 Laptop	3,000	5	0.65 [0.26, 1.25]	1.19 [1.18, 1.19]	0.96 [0.48, 1.56]	1.07 [1.02, 1.09]
RTX 4050 Laptop	10,000	5	0.46 [0.39, 0.54]	1.19 [1.19, 1.20]	0.97 [0.79, 1.10]	1.08 [1.06, 1.17]
RTX 4050 Laptop	30,000	5	0.59 [0.57, 0.68]	1.00 [1.00, 1.00]	0.95 [0.92, 1.01]	1.07 [1.02, 1.11]
RTX 4090 pods B	100	5	0.92 [0.91, 1.01]	1.00 [1.00, 1.00]	0.97 [0.95, 1.11]	1.07 [0.95, 1.08]
RTX 4090 pods B	1,000	5	0.90 [0.70, 0.92]	1.00 [1.00, 1.00]	0.97 [0.94, 1.03]	1.07 [1.06, 1.28]
RTX 4090 pods B	3,000	5	0.87 [0.35, 0.94]	1.00 [1.00, 1.00]	0.96 [0.54, 0.99]	1.07 [1.02, 1.07]
RTX 4090 pods B	10,000	5	0.82 [0.34, 0.91]	1.00 [1.00, 1.01]	0.94 [0.64, 1.00]	1.07 [1.07, 1.37]
RTX 4090 pods B	30,000	5	1.00 [0.46, 1.00]	1.00 [1.00, 1.00]	1.07 [0.57, 1.09]	1.07 [1.03, 1.16]
H100 SXM	1,000	3	0.86 [0.86, 0.88]	1.36 [1.35, 1.36]	1.24 [1.21, 1.25]	1.26 [1.26, 1.28]
H100 SXM	3,000	3	0.71 [0.65, 0.77]	1.35 [1.35, 1.35]	1.10 [1.03, 1.17]	1.27 [1.26, 1.28]
H100 SXM	10,000	3	0.81 [0.80, 0.88]	1.36 [1.36, 1.37]	1.18 [1.14, 1.48]	1.25 [1.19, 1.26]

before any cell completed; no completed cell was discarded. (iii) The pod first assigned to block 3 ran host-side work roughly $10\times$ slower than the others. At the user’s request, and before any of its results were inspected, block 3 was re-run from scratch on a new pod (Ryzen 9 7950X). The 14 cells the slow pod completed are retained (`dlr-b3-partial.tar.gz`) and excluded from the tables. A stray setup process ran for about 30 s on the block 4 pod during three cells; dropping those cells changes the affected light-path ratio from 0.967 to 0.966. (iv) The cross-host table (Table 1) and the device-time framing were not preregistered; they were suggested by the retained September 24 archive and are therefore partly exploratory, although the pods and H100 supplied new data for them.

B CPU/GPU CROSSOVER

Table 6: Median-block end-to-end P50 (ms) for the single-threaded CPU scan, the light GPU path with the heater off and on (winner against CPU in parentheses), and the heavy GPU path. The heater never changed a winner.

GPU	E	CPU	Light, heater off	Light, heater on	Heavy
RTX 4050 Laptop	100	0.010	0.172 (CPU)	0.253 (CPU)	1.999
RTX 4050 Laptop	1,000	0.083	0.230 (CPU)	0.374 (CPU)	2.050
RTX 4050 Laptop	3,000	0.308	0.548 (CPU)	0.559 (CPU)	2.109
RTX 4050 Laptop	10,000	3.332	1.682 (GPU)	1.685 (GPU)	2.204
RTX 4050 Laptop	30,000	9.187	2.265 (GPU)	2.138 (GPU)	2.791
RTX 4090 pods B	100	0.008	0.103 (CPU)	0.100 (CPU)	0.828
RTX 4090 pods B	1,000	0.048	0.123 (CPU)	0.123 (CPU)	0.835
RTX 4090 pods B	3,000	0.133	0.178 (CPU)	0.175 (CPU)	0.849
RTX 4090 pods B	10,000	1.495	0.430 (GPU)	0.405 (GPU)	0.860
RTX 4090 pods B	30,000	4.907	0.779 (GPU)	0.833 (GPU)	0.894

C THE HELD-OUT ROUTER FAILURE THAT MOTIVATED THIS STUDY

The system’s earlier research campaign (preserved unchanged in `paper/paper.tex`) selected a scalar routing rule, GPU if $E \cdot D \cdot B \geq 10^6$, with zero errors on 31 development CPU/GPU pairs. On a preregistered held-out suite of 16 workloads on an RTX 4050 laptop it chose the slower route in 8, with median regret 0.4% and maximum regret 235.7%. Each route was measured in a single process with five in-process runs. An audit of that archive found that the automatic arm differed from the forced arm of the same backend by more than 25% in 9 of 16 cells, and that two of the eight “wrong” labels were within 3.2% regret. In light of Section 4, labels near parity in such a suite partly measure the GPU’s power state during each process, and the per-workload count is not a stable statistic. The maximum-regret rejection survives: a $3.36\times$ ratio exceeds every within-host

spread we measured except the laptop light path’s $4.2\times$. The earlier manuscript also documents representation, preparation and harness findings that are not repeated here.

D COST AND RESOURCES

New cloud runs used five RTX 4090 pods (one block each), one H100 SXM pod and a 4-minute H100 probe, at \$0.74 and \$3.49 per hour. The operator’s ledger totals about US\$6 (`research/experiments/dvfs-dlr/campaign-log.md`). All pods were deleted after their data were downloaded and hash-checked. A container-level Vulkan fix (pointing the loader at `libEGLnvidia.so.0` rather than the toolkit’s GLX ICD) was required on every pod and is documented in the log; it probably explains an earlier “no Vulkan” failure on an A6000.